7. FRONTEND DEVELOPMENT

7.1 Tech Stack (Major Packages, Version, and Purpose)

The frontend is implemented as a React single-page application using TypeScript and Vite. The stack is selected for maintainability, fast development, and role-based UI separation.

Package	Version	Purpose in Project
react	^19.0.0	Core UI rendering and component model for all modules (Auth, Passenger, Driver, Admin).
react-dom	^19.0.0	Browser DOM renderer for React application mounting.
react-router-dom	^7.14.0	Route-level navigation, protected role routing, nested module routes.
axios	^1.15.0	Centralized HTTP client used by all API wrappers in frontend/src/api.
socket.io-client	^4.8.3	Real-time event communication for ride and status updates.
leaflet	^1.9.4	Interactive map engine for pickup-drop and route visualization.
react-leaflet	^5.0.0	React bindings for Leaflet map components.
lucide-react	^1.8.0	Icon library used in dashboards, forms, and navigation controls.
motion	^12.23.24	Screen transitions and entry animations for smoother UI interactions.
vite	^6.2.0	Build and dev server tooling with fast HMR and optimized builds.
typescript	~5.8.2	Static typing, safer API integration, and maintainable component contracts.
tailwindcss	^4.1.14	Utility-first styling framework used with custom design tokens.

7.2 UI Layout (Descriptive Implementation)

The frontend layout follows a role-first architecture:

1. **Global App Shell**
   - BrowserRouter is mounted in App.tsx.
   - Session token and user role are recovered from sessionStorage on load.
   - The role state controls route-level access and redirect behavior.

2. **Authentication Layout**
   - Auth module uses a centered, card-style form layout.
   - It includes three screens in a single page flow: login, signup, OTP verify.
   - Visual style uses glass panels, blurred accent gradients, and icon-led form controls.
3. **Passenger Layout**
   - Desktop: fixed left sidebar with major actions.
   - Mobile: bottom tab navigation for home, booking, trips, and profile.
   - Main panel hosts dashboard, booking, tracking, active ride, completion, history, and profile screens.
   - MapBackground overlays a live map with route polyline and dynamic markers.
4. **Driver Layout**
   - Desktop and mobile layouts mirror passenger structure for familiarity.
   - Includes ride request popup, navigation screen, active trip screen, document verification, and earnings views.
   - Real-time request acceptance is integrated into the active UI flow.
5. **Admin Layout**
   - Collapsible left sidebar + top header + content area.
   - Dedicated sections for dashboard KPIs, users, verification queue, revenue, and complaints.
   - Route-level rendering is handled via nested routes with animated transitions.

7.3 Navigation (All Available Routes)

7.3.1 Top-Level Routes

Route	Access Rule	Component
/login	Only when no authenticated role is active	AuthModule
/passenger/*	Requires role = passenger	PassengerModule
/driver/*	Requires role = driver	DriverModule
/admin/*	Requires role = admin	AdminModule
/	Redirects to /login or role root	Navigate

7.3.2 Passenger Sub-Routes

Route	Screen
/passenger/dashboard	PassengerDashboard
/passenger/book	BookRide

/passenger/searching	SearchingDriver
/passenger/tracking	RideTracking
/passenger/active	RideActive
/passenger/completion	RideCompletion
/passenger/history	RideHistory
/passenger/profile	PassengerProfile
/passenger	Redirect to dashboard

7.3.3 Driver Sub-Routes

Route	Screen
/driver/dashboard	DriverDashboard
/driver/navigation	NavigationScreen
/driver/active	RideActive
/driver/earnings	EarningsPage
/driver/verification	DriverVerification
/driver/profile	DriverProfile
/driver	Redirect to dashboard

7.3.4 Admin Sub-Routes

Route	Screen
/admin/dashboard	AdminDashboard
/admin/users	UserManagement
/admin/verification	DriverVerificationPanel
/admin/revenue	RevenuePanel
/admin/complaints	ComplaintsSystem
/admin	Redirect to dashboard

7.4 Components (React Implementation)

7.4.1 Shared UI Components

Component	Responsibility
GlassCard	Reusable glassmorphism container used across auth and dashboards.
Button	Unified button variants (primary, outline, danger, ghost).
Input	Label + optional icon + normalized input styles and spacing.
ScreenTransition	motion-based route/screen transition animation wrapper.
MapBackground	Full-screen map renderer with route line and marker overlays.
GeocodingAutocomplete	Debounced location search using Nominatim API with selection callback.

7.4.2 Page-Level Containers

- Auth.tsx: authentication flow and OTP verification state machine.
- Passenger.tsx: route container and socket listeners for passenger ride status events.
- Driver.tsx: route container, ride request popup orchestration, driver response actions.
- Admin.tsx: route container with sidebar-driven admin workflow.

7.4.3 Domain Sections

- Admin sections: AdminDashboard, UserManagement, DriverVerificationPanel, RevenuePanel, ComplaintsSystem.
- Driver sections: DriverDashboard, DriverProfile, DriverVerification, EarningsPage, NavigationScreen, RideActive, RideRequestPopup.

7.4.4 API Layer Components

- config.ts: centralized Axios instance and Authorization interceptor.
- auth.ts, rides.ts, driver.ts, admin.ts, verification.ts: feature-specific API wrappers.
- socket.ts: singleton socket service managing connect, emit, listener registry, and disconnect.

7.5 CSS / Bootstrap Styling

This frontend does not use Bootstrap classes or Bootstrap JS. Instead, it uses Tailwind CSS v4 with custom CSS utilities.

Styling system details:

- Tailwind import and theming in index.css.
- Custom design tokens via theme variables:
  - primary color = #FFD600
  - dark background = #0B0B0B
- Utility classes are used for spacing, typography, responsive breakpoints, and colors.
- Custom reusable classes include:
  - glass for cards
  - glass-panel for sidebars and headers
  - map-grid for optional map-style backgrounds

The resulting UI style is dark, high-contrast, role-oriented, and optimized for desktop plus mobile navigation patterns.

8. BACKEND DEVELOPMENT

8.1 Server Setup

8.1.1 Major Backend Packages, Version, and Purpose

Package	Version	Purpose in Project
express	^4.21.2	HTTP server framework, route mounting, middleware pipeline.
mongoose	^8.7.1	MongoDB ODM, schema modeling, validations, query abstraction.
dotenv	^16.4.5	Loads environment variables from .env at startup.
cors	^2.8.5	Cross-origin request control for frontend-backend communication.
helmet	^8.0.0	Security headers for safer HTTP responses.
morgan	^1.10.0	Request logging (dev/production modes).
express-rate-limit	^7.4.1	API throttling to reduce abuse and accidental flooding.
jsonwebtoken	^9.0.2	JWT token generation and verification for auth.
socket.io	^4.8.0	Real-time event transport for ride workflows.
multer	^2.1.1	Multipart upload handling for verification documents.
nodemailer	^8.0.5	OTP and status email delivery.
nodemon	^3.1.4	Auto-restart during development.
jest + supertest	^30.3.0 / ^7.2.2	Automated API and integration testing.

8.1.2 Dotenv and Configuration

Environment variables are loaded in config/env.js using dotenv.config().

Key variables:

- NODE_ENV
- PORT
- MONGO_URI
- JWT_SECRET
- CORS_ORIGIN
- ADMIN_USER
- ADMIN_PASS
- SMTP_HOST
- SMTP_PORT
- SMTP_USERNAME
- SMTP_PASSWORD
- EMAIL_FROM

8.1.3 Server Initialization Sequence

1. Environment bootstrap

- config/env.js loads and parses environment values.

2. App creation

- app.js creates Express app.
- Security and utility middleware added: helmet, cors, morgan, express.json, express.urlencoded.
- Health endpoint mounted at /health.
- Rate limiter mounted under /api except in test mode.
- Static upload directory exposed at /uploads.
- API router mounted under /api.
- 404 handler and centralized errorHandler registered.

3. HTTP + Socket server

- index.js creates HTTP server from app.
- initSocket(httpServer) initializes socket.io with CORS and namespace support.
- io instance attached to app context.

4. Database connection

- connectDB() validates MONGO_URI and connects to MongoDB.
- Mongoose connection events (connected/error/disconnected) are registered.

5. Server listen and lifecycle

- HTTP server listens on env.port.
- Graceful shutdown handles SIGINT and SIGTERM by closing server and disconnecting MongoDB.

8.2 Routing

Routing is organized by module in backend/src/routes and aggregated in routes/index.js.

Base API prefix: /api

Module routes:

- /api/auth
- /api/rides
- /api/users
- /api/drivers
- /api/passengers
- /api/ratings
- /api/earnings
- /api/verification
- /api/pricing
- /api/analytics
- /api/admin

Design pattern:

- Thin route files handle URL-to-controller mapping.
- Middleware is declared per route group or per route.
- Auth and role rules are enforced near the route definition for clarity.

8.3 Controllers

The controller layer handles HTTP concerns: request extraction, service invocation, status codes, and response formatting.

8.3.1 auth.controller.js

The authentication controller manages account creation, login, and OTP verification endpoints. The signup handlers for passengers and drivers forward validated payloads to the auth service and return creation responses that include OTP flow context (such as userId and email). The login and adminLogin handlers verify credentials through the service layer and return authenticated user metadata with JWT access tokens. The verifyOtpController finalizes the onboarding flow by validating the OTP, activating the account, and issuing a token for immediate authorized use.

8.3.2 rides.controller.js

The rides controller acts as the transport layer for the full ride lifecycle. It exposes fare estimation, booking, ride retrieval, status updates, cancellation, shared-group lookup, and user-specific ride history operations. It enriches requests with authenticated user context (for example passengerId from token), applies lifecycle-aware response codes, and normalizes outbound response objects for frontend consumption. This controller is the main bridge between client workflows and ride domain rules implemented in rides.service.js.

8.3.3 users.controller.js

The users controller provides profile-oriented account operations. It supports admin-level user listing, authenticated self-profile retrieval, and controlled profile updates for the currently logged-in user. The controller keeps response formatting consistent with success flags and data envelopes, while delegating validation and persistence rules to users.service.js.

8.3.4 drivers.controller.js

The drivers controller handles driver-facing and discovery-oriented endpoints. It includes profile retrieval/update operations for authenticated drivers, availability toggling (online/offline behavior), and generic driver listing/detail endpoints used by other actors. By separating these concerns from ride booking endpoints, the design keeps profile management and operational ride actions modular.

8.3.5 passengers.controller.js

The passengers controller encapsulates passenger profile and history concerns. It supports admin views for passenger listing/detail and provides a role-restricted endpoint for passengers to fetch their own ride history. The controller is intentionally lightweight and primarily composes filtered data returned by passengers.service.js.

8.3.6 pricing.controller.js

The pricing controller exposes fare and surge management interfaces. It supports pre-trip estimate generation, post-trip final fare calculation, and surge multiplier configuration/lookup by area code or coordinates. The controller coordinates payload normalization (pickup/drop aliases), throws explicit input errors for missing geospatial params, and returns pricing domain objects generated by pricing and analytics services.

8.3.7 verification.controller.js

The verification controller manages driver document compliance flows end-to-end. It processes multipart uploads, extracts document metadata, and creates verification records for the authenticated driver. It also provides admin queue visibility and moderation endpoints for approval/rejection decisions, including reason-based rejection handling. This controller is central to onboarding trust, safety enforcement, and driver activation governance.

8.3.8 earnings.controller.js

The earnings controller handles driver income visibility and payout operations. It provides period-based earnings summaries, paginated trip earnings lists, and admin-level payout management endpoints. It also normalizes response payloads for UI compatibility (for example converting internal items arrays into trips arrays where required by frontend/tests).

8.3.9 ratings.controller.js

The ratings controller governs feedback capture and retrieval. It accepts rating submissions tied to rides and exposes read endpoints for driver-level rating summaries and ride-specific feedback lookup. It acts as a clean API boundary over rating validation and aggregation logic handled by ratings.service.js.

8.3.10 analytics.controller.js

The analytics controller provides operational intelligence endpoints for administrators. It supports ride/revenue trend retrieval, demand-zone aggregation triggers, current-hour heatmap views, historical zone analysis, and dataset export. This controller is designed for dashboard and reporting needs, with strict admin access and query-driven analytics filtering.

8.3.11 admin.controller.js

The admin controller centralizes platform governance operations. It handles high-level dashboard KPIs, user state controls (suspend/activate/block/unblock/delete), ride oversight, driver listing, complaint retrieval, and admin-facing detail views. The controller standardizes privileged operations behind role checks and acts as the orchestrator for cross-domain administrative actions.

8.4 Business Logic (Services)

Services implement domain rules, transitions, calculations, and database operations.

8.4.1 auth.service.js

The authentication service implements core identity logic, including secure registration, credential verification, and token issuance. During signup, it enforces unique emails, hashes passwords, creates pending users, generates OTP codes with expiry windows, and triggers transactional email delivery. During login, it validates credentials, checks account state (active/blocked), and issues JWT tokens with role-aware claims. It also supports OTP-based account activation and includes an admin authentication path backed by either database records or configured environment credentials.

8.4.2 rides.service.js

The rides service is the largest business module and manages ride state from request to completion. It implements status-transition rules, nearest-driver search, timed no-driver handling, assignment workflows, driver acceptance/rejection handling, cancellation authorization, and location-event processing. It also emits role-specific socket events, integrates with pricing and analytics helpers, and stores lifecycle history for auditability. By encoding transition constraints in one service, it prevents invalid ride movements and keeps workflow behavior consistent across HTTP and socket triggers.

8.4.3 pricing.service.js

The pricing service models fare computation as a composable calculation pipeline. It estimates min/max fare using base fare, distance/time projections, waiting thresholds, ride type discount rules, and surge multipliers. For completed trips, it computes final billing from actual location logs and duration data. It also provides administrative surge configuration per zone and time-slot, enabling demand-responsive pricing behavior.

8.4.4 verification.service.js

The verification service governs driver onboarding compliance and document review state. It tracks required document categories, computes completeness/rejection/approval summaries, and updates driver verification status based on review outcomes. Approval flows enforce completion prerequisites, while rejection flows capture reasons and restrict availability as needed. It also pushes role-based notifications and email updates so drivers and admins receive timely verification state changes.

8.4.5 analytics.service.js

The analytics service provides decision-support logic for operational monitoring. It aggregates rides into geohash-based demand zones, computes temporal demand metrics, and returns heatmap/history views used by admin dashboards. It also exposes ride and revenue aggregate calculations plus exportable datasets for downstream analysis or ML workflows. This service transforms raw transactional ride data into management-ready intelligence.

8.4.6 admin.service.js

The admin service implements privileged platform-management operations. It computes summary KPIs, handles user lifecycle controls, provides filtered ride and driver

management datasets, and surfaces complaint-style signals from ratings data. It additionally returns enriched ride detail snapshots (fare, rating, log counts) required for investigation and support workflows.

8.4.7 drivers.service.js

The drivers service is focused on driver profile and availability domain logic. It supports listing and lookup operations for discovery use-cases while exposing authenticated profile update actions for driver self-management. Availability toggles are funneled through this service to keep online/offline state transitions explicit and reusable.

8.4.8 passengers.service.js

The passengers service provides passenger-oriented read operations required by both self-service and admin workflows. It handles passenger profile retrieval and aggregates historical ride records for timeline-style views. The service isolates passenger data access rules from controller and routing concerns.

8.4.9 earnings.service.js

The earnings service translates completed ride economics into payout-ready records and summary metrics. It calculates period-scoped gross, commission, net, pending payout totals, and paginated earning trip lists for driver dashboards. For administrators, it groups pending payouts and supports mark-paid transitions, enabling controlled payout operations.

8.4.10 ratings.service.js

The ratings service applies business rules for ride feedback persistence and retrieval. It validates rating eligibility, records review content, and computes driver-level aggregate summaries used in trust and quality indicators. It also supports per-ride feedback retrieval for dispute analysis and performance review.

8.4.11 users.service.js

The users service provides common account-management behavior across roles. It supports paginated user listing with optional filters, authenticated profile retrieval, and constrained profile updates for permitted fields. By centralizing these operations, the service reduces duplication between admin and self-service flows.

8.4.12 mail.service.js

The mail service abstracts SMTP transport setup and outbound transactional messaging. It is used primarily for OTP delivery and account/verification status communication. Encapsulating email dispatch in a dedicated service keeps communication concerns separate from business logic while enabling easier provider or template changes.

8.5 Middleware

Middleware	Primary Role
protect / authCheck	Validates Bearer JWT, decodes req.user, returns 401 on invalid token.
socketAuth	Validates socket auth token and attaches user context to socket.
authorize / roleGuard	Enforces role-based access; returns 403 on role mismatch.
validateRequired	Generic required-field validator for request body/query sources.
upload (multer)	Handles document uploads, file type filter, file size limit (5MB).
errorHandler	Centralized error response builder with stack in non-production.

8.6 Authentication

Authentication model in this backend:

- **Type:** JWT Bearer token authentication.
- **Token generation:** generateAccessToken(payload, expiresIn = 7d).
- **Token payload:** userId, role, and optional claims (example: isVerifiedDriver).
- **Transport:** Authorization: Bearer <token> for HTTP, auth.token for socket handshake.
- **Authorization:** roleGuard with allowed roles per route.
- **Account states:** active, pending, blocked (used in login and access checks).
- **OTP workflow:** signup creates pending user + OTP, verify-otp activates account and returns token.

9. API DESIGN AND INTEGRATION

9.1 API Conventions

9.1.1 Base and Protocol Conventions

- Base URL: http://localhost:5000/api
- Data format: JSON (multipart/form-data for document upload)
- Authentication header: Authorization: Bearer <token>
- Default content type: application/json
- Time/resource identifiers: MongoDB ObjectId style IDs

9.1.2 Status Code Conventions

- 200: Success for reads and many updates
- 201: Resource created
- 202: Accepted state (used when booking can continue but no immediate driver)
- 400: Validation/input errors
- 401: Unauthorized (missing/invalid token)
- 403: Forbidden (insufficient role)
- 404: Not found
- 409: Conflict (example: duplicate email)
- 500: Internal server error

9.1.3 Response Envelope (Documentation Standard)

Success envelope sample:

```
{
  "success": true,
  "message": "Operation completed",
  "data": {}
}
```

Error envelope sample:

```
{
  "success": false,
  "message": "Validation failed",
  "error": {
    "code": "BAD_REQUEST",
    "details": []
  }
}
```

Note: This project already uses success/message patterns in most modules; some analytics/pricing endpoints return direct domain objects. A single envelope can be adopted at integration gateway level if strict uniformity is required.

9.2 Endpoint Tables (All Route Endpoints)

9.2.1 Utility Endpoints

Method	Endpoint	Auth Required	Role	Description
GET	/health	No	Public	Backend health check.

9.2.2 Authentication Endpoints

Method	Endpoint	Auth Required	Role	Description
POST	/api/auth/signup/passenger	No	Public	Register passenger and send OTP.
POST	/api/auth/signup/driver	No	Public	Register driver and send OTP.
POST	/api/auth/login	No	Public	Login user and return token.
POST	/api/auth/admin/login	No	Public/Admin	Admin login endpoint.
POST	/api/auth/verify-otp	No	Public	Verify OTP and activate account.

9.2.3 Ride Endpoints

Method	Endpoint	Auth Required	Role	Description
POST	/api/rides/estimate	No	Public	Estimate fare and trip details.
POST	/api/rides/book	Yes	Passenger	Book ride request.
GET	/api/rides/shared/:groupId	Yes	Any authenticated	Fetch shared ride grouping data.
GET	/api/rides/history	Yes	Any authenticated	Fetch requesting user ride history.
GET	/api/rides/:rideId	Yes	Any authenticated	Fetch ride by ID.
POST	/api/rides/:rideId/status	Yes	Driver	Update ride status (alias).
PATCH	/api/rides/:rideId/status	Yes	Driver	Update ride status.
POST	/api/rides/:rideId/cancel	Yes	Passenger or Driver	Cancel ride by actor.
POST	/api/rides/:rideId/mock-assign	Yes	Admin	Test/admin mock assignment endpoint.
POST	/api/rides/:rideId/mock-driver-action	Yes	Driver	Test alias for driver status action.

9.2.4 User Endpoints

Method	Endpoint	Auth Required	Role	Description
GET	/api/users	Yes	Admin	List users.
GET	/api/users/me	Yes	Any authenticated	Fetch own profile.
PATCH	/api/users/me	Yes	Any authenticated	Update own profile fields.

9.2.5 Driver Endpoints

Method	Endpoint	Auth Required	Role	Description
GET	/api/drivers/me	Yes	Driver	Fetch driver profile for logged-in driver.
PATCH	/api/drivers/me/profile	Yes	Driver	Update driver profile details.
PATCH	/api/drivers/me/availability	Yes	Driver	Update online/availability status.
GET	/api/drivers	Yes	Any authenticated	List drivers (with filters).
GET	/api/drivers/:driverId	Yes	Any authenticated	Fetch driver by ID.

9.2.6 Passenger Endpoints

Method	Endpoint	Auth Required	Role	Description
GET	/api/passengers	Yes	Admin	List passengers.
GET	/api/passengers/me/rides	Yes	Passenger	Passenger self ride history.
GET	/api/passengers/:passengerId	Yes	Admin	Fetch passenger by ID.

9.2.7 Ratings Endpoints

Method	Endpoint	Auth Required	Role	Description
POST	/api/ratings	Yes	Any authenticated	Submit ride rating.
GET	/api/ratings/driver/:driverId	No	Public	Driver ratings + summary.
GET	/api/ratings/ride/:rideId	No	Public	Ratings by ride.

9.2.8 Earnings Endpoints

Method	Endpoint	Auth Required	Role	Description
GET	/api/earnings/summary	Yes	Driver	Driver earnings summary by period.
GET	/api/earnings/trips	Yes	Driver	Driver earning trip list (paginated).

9.2.9 Verification Endpoints

Method	Endpoint	Auth Required	Role	Description
POST	/api/verification/upload	Yes	Driver	Upload verification document.
GET	/api/verification/status	Yes	Driver	Fetch own verification status summary.

9.2.10 Pricing Endpoints

Method	Endpoint	Auth Required	Role	Description
POST	/api/pricing/estimate	No	Public	Estimate fare from coordinates.
POST	/api/pricing/final/:rideId	Yes	Driver or Admin	Calculate and finalize ride fare.
POST	/api/pricing/surge	Yes	Admin	Set surge multiplier by area.
GET	/api/pricing/surge/:areaCode	No	Public	Get surge by area code.
GET	/api/pricing/surge?lat=&lng=	No	Public	Get surge by coordinates.

9.2.11 Analytics Endpoints

Method	Endpoint	Auth Required	Role	Description
GET	/api/analytics/rides	Yes	Admin	Ride analytics data.
GET	/api/analytics/revenue	Yes	Admin	Revenue analytics data.
POST	/api/analytics/aggregate	Yes	Admin	Trigger zone aggregation job.
GET	/api/analytics/heatmap	Yes	Admin	Current-hour demand heatmap.
GET	/api/analytics/history	Yes	Admin	Zone demand history by time range.
GET	/api/analytics/export	Yes	Admin	Export analytics dataset.

9.2.12 Admin Endpoints

Method	Endpoint	Auth Required	Role	Description
GET	/api/admin/dashboard	Yes	Admin	Dashboard KPI summary.
PATCH	/api/admin/users/:userId/suspend	Yes	Admin	Suspend user with reason.
PATCH	/api/admin/users/:userId/activate	Yes	Admin	Reactivate suspended user.
GET	/api/admin/verification/queue	Yes	Admin	Pending verification queue.
PATCH	/api/admin/verification/:driverId/approve	Yes	Admin	Approve driver verification.
PATCH	/api/admin/verification/:driverId/reject	Yes	Admin	Reject verification with reason.
GET	/api/admin/users	Yes	Admin	Admin user listing with filters.
PATCH	/api/admin/users/:userId/block	Yes	Admin	Block user account.
PATCH	/api/admin/users/:userId/unblock	Yes	Admin	Unblock user account.
DELETE	/api/admin/users/:userId	Yes	Admin	Delete user account.
GET	/api/admin/rides	Yes	Admin	Admin ride listing.
GET	/api/admin/rides/:rideId	Yes	Admin	Ride detail for admin view.
GET	/api/admin/drivers	Yes	Admin	Driver listing for admin.
GET	/api/admin/complaints	Yes	Admin	Complaint list (rating-based).
GET	/api/admin/earnings/payouts	Yes	Admin	Pending payout groups.
PATCH	/api/admin/earnings/:earningsId/mark-paid	Yes	Admin	Mark payout item as paid.

9.3 Request/Response Formats (5 Samples)

Sample 1: Passenger Signup

Request

POST /api/auth/signup/passenger

```
{"name": "Neha Sharma",
 "email": "neha@example.com",
 "phone": "9876543210",
 "password": "Pass@1234"}
```

Response (201)

```
{ "success": true,
  "message": "OTP sent to your email",
  "userId":"67fce1234abcde0011223344",
  "email": "neha@example.com"
}
```

Sample 2: OTP Verification

Request

POST /api/auth/verify-otp

```
{
  "userId":
"67fce1234abcde0011223344",
  "otp": "482901"
}
```

Response (200)

```
{
  "success": true,
  "user": {
    "id": "67fce1234abcde0011223344",
    "name": "Neha Sharma",
    "email": "neha@example.com",
    "role": "passenger",
    "phone": "9876543210",
    "status": "active"
  },
  "token": "eyJhbGciOiJI..."
}
```

Sample 3: Ride Booking (Passenger)

Request

POST /api/rides/book

```
{
  "pickup": {
    "address": "College Gate",
    "lat": 12.9716,
    "lng": 77.5946
  },"drop": {
    "address": "City Mall",
    "lat": 12.9352,
    "lng": 77.6245
  },
  "rideType": "solo"}
```

Response (201)

```
{
  "success": true,
  "rideId": "67fce5aa11bb22cc33dd44ee",
  "driver": null,
  "passenger":
"67fce1234abcde0011223344",
  "fare": 182.4,
  "status": "searching_driver"
}
```

Sample 4: Upload Driver Verification Document

Request

POST /api/verification/upload
Content-Type: multipart/form-data

form-data:
- documentType: license
- document: <binary-file>

Response (201)

```
{
  "success": true,
  "driverId":"67fceabcd112233445566778",
  "documentType": "license",
  "fileRef": "/uploads/verification/document-
1712300000-123456789.png",
  "reviewStatus": "pending"
}
```

Sample 5: Driver Earnings Trips

Request
GET /api/earnings/trips?page=1&limit=20

Response (200)

```
{
  "trips": [
    {
      "_id": "67fce9aabbccddeeff001122",
      "rideId": {
        "_id": "67fce5aa11bb22cc33dd44ee",
        "status": "trip_completed",
        "finalFare": 190
      },
      "grossAmount": 190,
      "netAmount": 152
    }
  ],
  "page": 1,
  "limit": 20,
  "total": 8
}
```


9.4 Additional JSON Payload Samples

9.4.1 Set Surge (Admin)

POST /api/pricing/surge

```
{
  "areaCode": "tdr1v2",
  "multiplier": 1.4,
  "timeSlot": 18,
  "dayOfWeek": 5
}
```


9.4.2 Reject Verification (Admin)

PATCH /api/admin/verification/:driverId/reject

```
{
  "documentType": "insurance",
  "reason": "Document is blurry and expiry date not visible"
}
```

9.4.3 Standard Error Format from Validation Middleware

```
{
  "success": false,
  "message": "Missing required fields: email, password"
}
```

9.5 Integration Flow

9.5.1 Frontend-Backend REST Flow

1. Authentication flow

- Auth screen submits credentials to auth API wrapper.
- Axios instance attaches token automatically after login.
- Token and user data are persisted in sessionStorage.
- App role state redirects user to role-specific module routes.

2. Passenger booking flow

- Passenger book screen collects pickup/drop via geocoding + map.
- Request sent to /api/rides/estimate for pre-booking fare preview.
- Booking request sent to /api/rides/book with bearer token.
- Ride ID is persisted for further tracking.

3. Driver workflow

- Driver updates availability/profile via /api/drivers endpoints.
- Earnings screens consume /api/earnings/summary and /api/earnings/trips.
- Verification upload uses multipart endpoint /api/verification/upload.

4. Admin workflow

- Admin dashboard fetches /api/admin/dashboard KPIs.
- Verification queue and decisions use /api/admin/verification/* endpoints.
- User operations (block/unblock/suspend/delete) use /api/admin/users/*.
- Revenue and complaint views consume dedicated admin analytics endpoints.

9.5.2 Real-Time Socket Integration Flow

1. Connection phase

- Frontend socket service connects with auth token in handshake.
- Backend validates token through socketAuth middleware.
- User socket is registered in registry maps by user ID and role.

2. Driver matching and assignment

- Passenger books ride through REST.
- Backend emits driver:new_ride_request to selected nearby driver.
- Driver module shows RideRequestPopup and emits driver:ride_response.
- Backend updates ride and emits passenger:ride_status and passenger:driver_assigned.

3. Ride tracking lifecycle

- Driver emits driver:location_update and ride:status_update.
- Backend processes updates and forwards role-specific events.
- Passenger UI transitions through searching, tracking, active, and completion screens.
- Admin receives feed events for operations monitoring.

4. Verification and admin events

- Driver document upload triggers admin queue visibility.
- Admin approval/rejection updates driver state and can trigger notifications.

9.5.3 Integration Reliability Controls

- Axios interceptor enforces centralized token propagation.
- Role-protected routing prevents unauthorized module rendering.
- Middleware-based auth/role checks mirror frontend route guards.
- Error handler centralizes backend failure responses.
- Socket listener registry in frontend prevents lost listeners during reconnect cycles.